



LAB PRACTICE № 3: OBJECTS, CONSTRUCTORS, STATIC MEMBERS AND INHERITANCE

COMP1020: OOP, Algorithms, and Data Structures

Week 03

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your classes and methods should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables, methods and classes should have meaningful names, and should be followed Java programming convention.
- Academic honesty is required in all work you submit to be graded. Although you can discuss among yourself at high level how to solve the lab problem, you should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions.
- You should upload your *.java file(s) to Canvas **before the end of the laboratory session** unless the instructor gave a specified deadline. No Canvas submission will get 0 point (of course).
- Submit separate *.java file for each Lab problem with the exact file name specified in each lab question (do not add your student ID or your name). For example: **HelloWorld.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
 - (a) No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 - (b) The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 - (c) The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

stdin	stdout
3 5	15 16

Figure 1: Example testcase of Problem 01

Learning Outcome

Topics: Objects, constructors, constructor calls, static members and inheritance in Java.

Outcome: Acquire the skill of creating own objects, using them from other classes. Be able to create multiple constructors for a class and understand constructor calls in inherited classes. Acquire the skills of creating and using static class members.

Problem 01: Area and Perimeter of a rectangle

Write a program and name it `Rectangle.java`: Given a rectangle, compute the area and the perimeter of this rectangle.

- Design a class representing rectangles with 2 properties: length, width.
- Use constructor and 2 operations: `computeArea()` and `computePerimeter()`.
- Read 2 integers L and W from standard input and create a rectangle with length L and width W , compute the area and the perimeter of that rectangle.

Input

- Line 1 contains 2 integers L and W ($1 \leq L, W \leq 1000$)

Output

- The area and perimeter of the rectangle.

Example See Figure 1

Problem 02: Matrix manipulation

Write a program and name it `MatrixManipulation.java`: Design a class representing a matrix of integers with the following properties:

- row and column: number of rows and columns
- matrix: the 2-dimensional array representing the matrix (rows are indexed $1, 2, \dots, \text{row}$ and columns are $1, 2, \dots, \text{column}$)

The class features operations:

- `int sum()`: return the sum of elements of the matrix m
- `int subsum(int a, int b, int c, int d)`: sum of items from row a to row b and column c to column d
- `void update(int r, int c, int v)`: `matrix[r][c]` is updated with the new value v

Write a program that reads the matrix from stdin and performs a sequence of operations described above.

Input

stdin	stdout
3 4	25
1 2 3 4	16
1 1 1 2	
3 2 4 1	
sum	
update 1 2 5	
subsum 1 2 2 4	
#	

Figure 2: Example testcase of Problem 02

- Line 1: contains 2 positive integers m and n which are number of rows and columns ($1 \leq m, n \leq 100$)
- Line $i + 1$ ($i = 1, \dots, m$): contains the i^{th} row of the matrix
- Subsequent lines are operations, each line contains the information of an operation with the format below:
 - sum (return the sum of items of the matrix)
 - subsum a b c d (return the sum of items from row a to row b and column c to column d)
 - update r c v (update the item $matrix[r][c]$ with the new value v)
- The input is terminated with a line containing #.

Output

- Write in each line the result of the corresponding operation (sum or subsum)

Example See Figure 2

Problem 03: Salary

Write a program and name it `Salary.java`: The HR department of a university need to manage the salary of employees. There two kinds of employees: staff and teacher. Each employees has an identifier (ID). The salary of a staff depends on an indicator called PV: salary of a staff = $1000 \cdot PV$. For teachers, apart from the PV, the salary of a teacher depends also on another indicator called GD: salary of a teacher = $GD \cdot 2000 + PV \cdot 1500$. The indicators of staffs and teachers might change over times.

Design a class `Staff` representing the staff with

- Two properties: ID and PV which are integers
- Two methods: `void updatePV(int PV); int getSalary()`
- The constructor: `Staff(int ID, int PV)`

Design a class `Teacher` that extends the class `Staff` with an additional property which is GD (integer) overrides the method `getSalary()` of the class `Staff`. The Constructor of class `Teacher` is as follows:
`public Teacher(int ID, int GD, int PV)`

Input The input consists of a sequence of lines, each line contains the information about an event:

- `TEACHER <ID> <PV> <GD>`: a new teacher joins the university with initial indicators: $PV = \langle PV \rangle$ and $GD = \langle GD \rangle$
- `STAFF <ID> <PV>`: a new staff joins the university with the initial indicator: $PV = \langle PV \rangle$

stdin	stdout
TEACHER 1 2 2	9000
TEACHER 2 2 3	5500
SALARY 2	7000
STAFF 3 6	13500
UPDATE 2 5	
UPDATE 1 1	
SALARY 1	
UPDATE 3 7	
SALARY 3	
SALARY 2	
#	

Figure 3: Example testcase of Problem 03

- UPDATE <ID> <NEW_PV>: change the indicator PV of the employee (having ID = <ID>) with new value <NEW_PV> (do nothing if the employee having ID = <ID> does not exist in the university).
- SALARY <ID>: return the current salary of the employee with ID = <ID> (return 0 if the employee having ID = <ID> does not exist in the university)
- The IDs of employees of the university are distinct.
- The input is terminated with a line containing #.

Output

- the result of the event: SALARY <ID> read from the input

Example See Figure 3